

Deep Learning – 1RT720

Report for Hand-in Assignment 2

Rafael Proença

March 3, 2026

1 Introduction

In this assignment I classify MNIST handwritten digits using PyTorch. Starting from the fully-connected network of Hand-in Assignment 1, I progressively build convolutional architectures, explore the Adam optimizer, add residual connections, and apply regularization (Batch Normalization and Dropout). All experiments were run on Google Colab with GPU (CUDA).

Generative AI disclosure: GitHub Copilot was used as a coding assistant for boilerplate, plotting, and markdown drafting. All architectural decisions, hyperparameter choices, and written analysis are my own.

The full code is available in Appendix [A](#).

2 Exercises

Exercise 1. Multi-layer fully-connected neural network

In Exercise 1 I re-implement the same fully-connected network from Hand-in 1 (Exercise 3, ReLU activation) using built-in PyTorch layers. The architecture is [784, 128, 64, 10] with ReLU hidden activations, trained with SGD (lr=0.1, batch size=64, 50 epochs).

(a) Comparison with HW1: The FC network achieves 100.00% train accuracy and 98.20% test accuracy (train cost 0.0005, test cost 0.0882) after 50 epochs. This is consistent with the Hand-in 1 results, where the same architecture produced comparable accuracy (98.04% test). The large gap between train cost (≈ 0) and test cost (0.0882) indicates overfitting, which is expected for a fully-connected network with 109,386 parameters trained for 50 epochs on MNIST without regularization.

(b) Speed comparison: Training took 7 min 4 sec on GPU (Google Colab with CUDA), compared to 1 min 13 sec for the NumPy implementation in Hand-in 1 — making PyTorch roughly $6\times$ slower for this particular experiment. Part of this difference comes from the full-dataset test evaluation performed every epoch (vs. mini-batch evaluation in HW1), but even accounting for that, PyTorch is slower here because the GPU kernel-launch overhead and CPU $\leftrightarrow$ GPU data transfers outweigh the parallelism benefit when the matrices are small (the largest layer is only 784×128). For the larger convolutional networks in later exercises, GPU acceleration becomes worthwhile.

(c) Learning curve: See Figure 1.

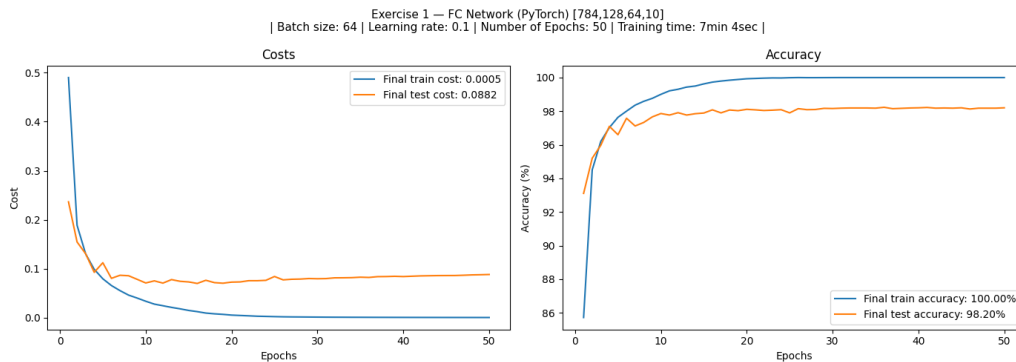


Figure 1: Exercise 1 — Learning curves for the fully-connected network [784,128,64,10] with ReLU and SGD. The model converges to 100% train accuracy but the rising test cost indicates overfitting.

Exercise 2. Multi-layer convolutional neural network

In Exercise 2 I replace the fully-connected architecture with the CNN specified in the assignment (three convolutional layers with ReLU and max pooling, followed by a linear classifier), trained with SGD (lr = 0.01, batch size = 64, 20 epochs).

(a) **Parameter count:** Table 1 shows the layer-by-layer parameter count.

Table 1: Layer-wise parameter count for the CNN (Exercise 2).

Layer	Calculation	Parameters
conv1 ($1 \rightarrow 8, 3 \times 3$)	$(3 \times 3 \times 1) \times 8 + 8$	80
conv2 ($8 \rightarrow 16, 3 \times 3$)	$(3 \times 3 \times 8) \times 16 + 16$	1,168
conv3 ($16 \rightarrow 32, 3 \times 3$)	$(3 \times 3 \times 16) \times 32 + 32$	4,640
fc ($1568 \rightarrow 10$)	$(32 \times 7 \times 7) \times 10 + 10$	15,690
Total		21,578

The CNN uses $5.1\times$ fewer parameters than the FC network (109,386), yet achieves better generalization thanks to weight sharing in convolutions.

(b) **Learning curve and observations:** The CNN achieves 98.65 % train / 98.20 % test accuracy after 20 epochs, surpassing the required 98 % threshold. The train–test gap is only 0.45 pp (vs 1.80 pp for the FC network), indicating much better generalization. This is expected since convolutions exploit the spatial structure of images and are less prone to memorizing individual pixel patterns. See Figure 2.

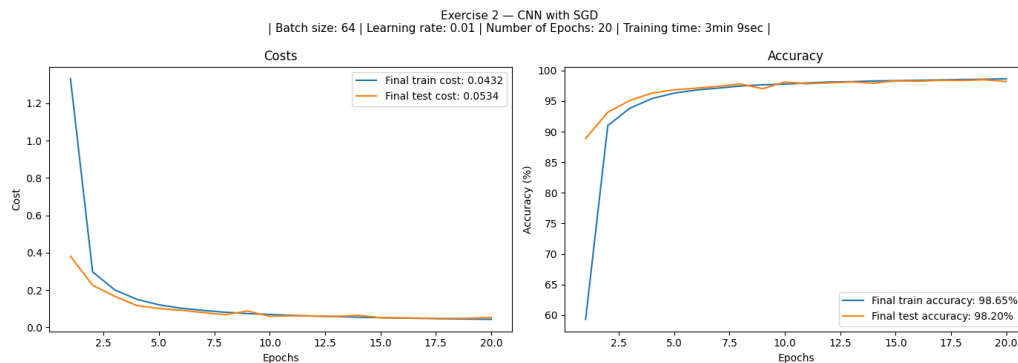


Figure 2: Exercise 2 — Learning curves for the CNN with SGD. The model reaches 98.20 % test accuracy with a small train–test gap, showing good generalization.

Exercise 3. CNN with Adam optimizer

In Exercise 3 I train the same CNN architecture but replace SGD with the Adam optimizer using default PyTorch parameters ($\beta_1=0.9$, $\beta_2=0.999$, $\varepsilon=10^{-8}$, $\text{lr}=0.001$).

(a) Observations: Adam converges much faster than SGD. By epoch 1, Adam already reaches 97.94 % test accuracy, whereas SGD needed several epochs to get past 90 %. Final results: Adam achieves 99.77 % train / 98.99 % test vs. SGD's 98.65 % train / 98.20 % test. However, the Adam model shows a wider train-test gap (0.78 pp vs 0.45 pp) and the test cost started rising after epoch 5, suggesting mild overfitting in later epochs. Training time was similar (~ 3 min each on GPU).

(b) Learning curve: See Figure 3.

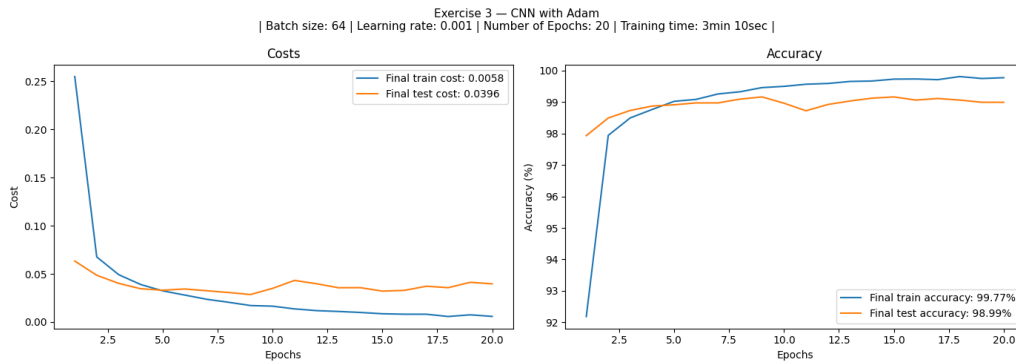


Figure 3: Exercise 3 — Learning curves for the CNN with Adam. Much faster convergence than SGD, reaching 98.99 % test accuracy.

Exercise 4. Residual connections

In Exercise 4 I add residual blocks (skip connections) to the CNN. Each residual block consists of two conv+ReLU pairs whose output is added to the block input: $\mathcal{F}(\mathbf{x}) + \mathbf{x}$. A residual block is placed after each convolutional stage, creating a deeper architecture (ResConvNet) with 45,882 parameters.

(a) Observations: The residual CNN achieves 99.78 % train / 99.15 % test accuracy, improving over Exercise 3's plain CNN (98.99 % test). The skip connections help gradient flow through the deeper network, leading to better optimization. Training took 4min 5sec on GPU (slightly longer due to the extra convolution layers in the residual blocks). The train-test gap is 0.63 pp and the test cost (0.0391) remains low, indicating good generalization even without explicit regularization.

(b) Learning curve: See Figure 4.

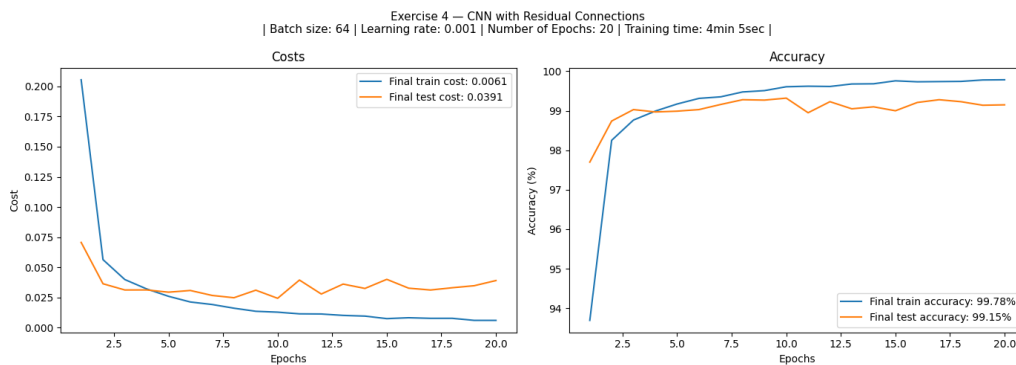


Figure 4: Exercise 4 — Learning curves for the ResConvNet with residual connections and Adam. Test accuracy reaches 99.15 %.

Exercise 5. Improve the CNN with Regularization

(a) Overfitting or underfitting? The Exercise 4 learning curves show mild overfitting. Train accuracy reaches 99.78 % while test accuracy is 99.15 %, giving a gap of 0.63 pp. The training loss (0.0061) is about $6.4\times$ lower than the test loss (0.0391), and the test loss curve stops decreasing and starts fluctuating around epoch 7–8 while the training loss continues to fall. Both metrics are high in absolute terms, so there is no underfitting — the network has sufficient capacity. Regularization techniques such as Batch Normalization or Dropout may help reduce the remaining generalization gap.

(b) Learning curves for BN and Dropout: See Figures 5 and 6.

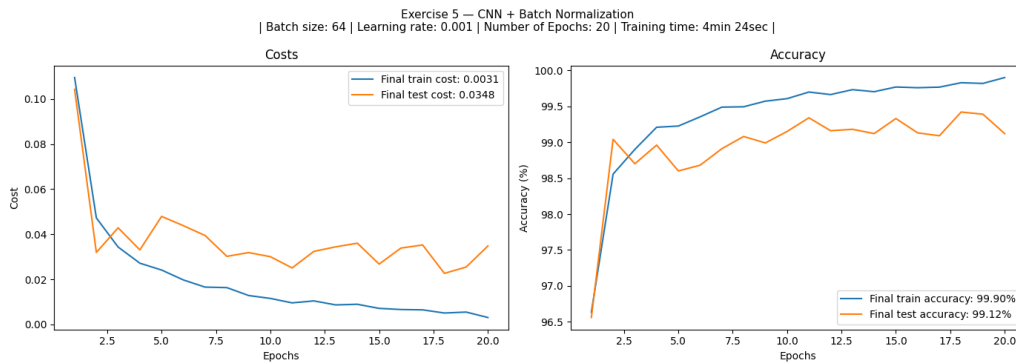


Figure 5: Exercise 5 — Learning curves for the ResConvNet with Batch Normalization. Test accuracy reaches 99.12%.

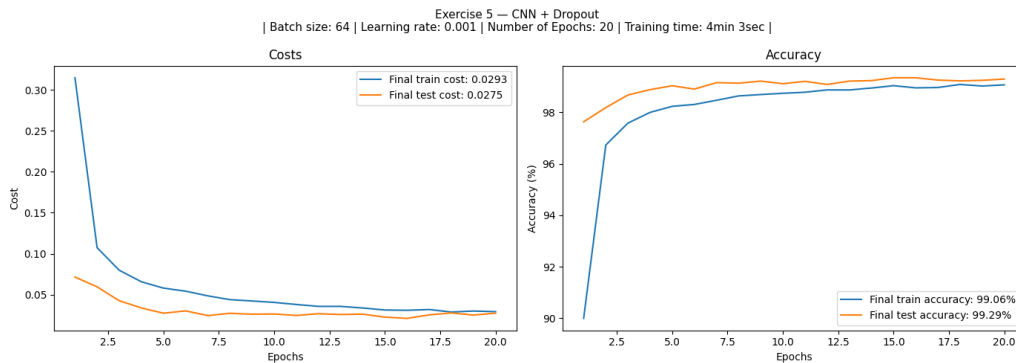


Figure 6: Exercise 5 — Learning curves for the ResConvNet with Dropout. Test accuracy reaches 99.29% with test cost lower than train cost.

(c) Comparison and confusion matrix: Table 2 compares the two regularized variants.

Table 2: Comparison of Batch Normalization and Dropout variants (Exercise 5).

Model	Train acc	Test acc	Train cost	Test cost	Time
BN	99.90 %	99.12 %	0.0031	0.0348	4 min 24 s
Dropout	99.06 %	99.29 %	0.0293	0.0275	4 min 3 s

Dropout wins with a test accuracy of 99.29 % vs. BN’s 99.12 %. The key difference is in generalization: Dropout’s test cost (0.0275) is *lower* than its train cost (0.0293), meaning the model generalizes excellently — the regularization effect of randomly dropping neurons prevents co-adaptation. BN, on the other hand, has a large train–test cost gap (0.0031 vs. 0.0348), indicating that it still overfits despite normalizing activations.

The confusion matrix for the Dropout model (Figure 7) shows near-perfect classification across all digits. The most common confusions are $7 \rightarrow 2$ (9 errors), $5 \rightarrow 3$ (7 errors), and $4 \rightarrow 9$ (5 errors), which are visually plausible since these digit pairs share similar stroke patterns.

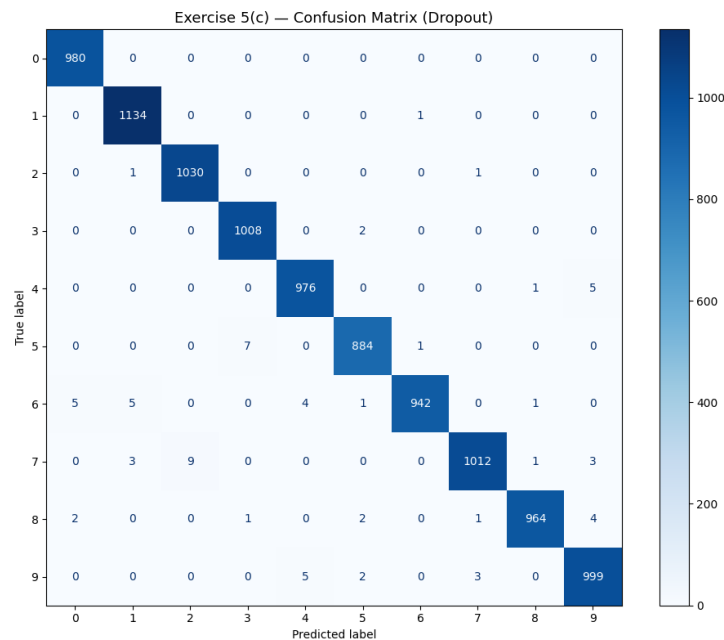


Figure 7: Exercise 5(c) — Confusion matrix for the Dropout model on the test set.

(d) Misclassified examples: Figure 8 shows 10 misclassified examples from the Dropout model, one per digit class where possible. The errors occur on ambiguous, poorly-written samples rather than systematic model failures.

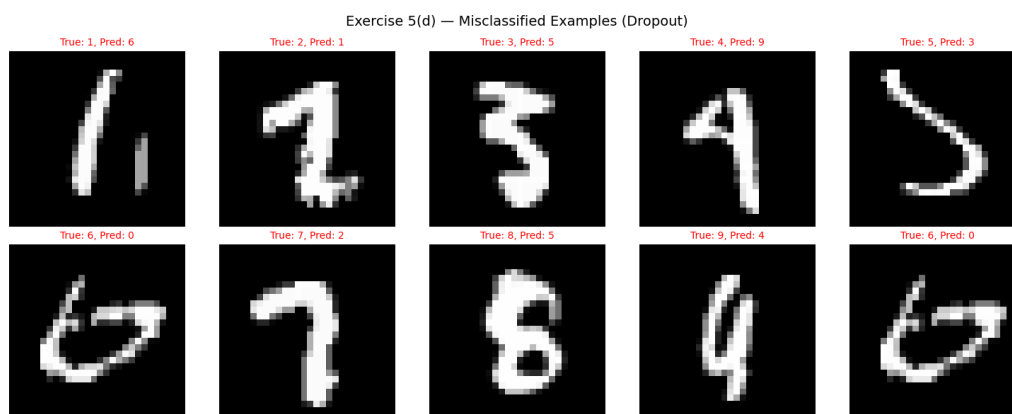


Figure 8: Exercise 5(d) — Ten misclassified examples from the Dropout model. Each image shows the ground-truth and predicted labels.

3 Summary

Table 3: Summary of all experiments.

Exercise	Model	Test Acc	Key Takeaway
1	FC [784, 128, 64, 10] + SGD	98.20 %	Baseline; overfits with 109 k params
2	CNN (3 conv) + SGD	98.20 %	5.1× fewer params, better generalization
3	CNN + Adam	98.99 %	Faster convergence, slight overfitting late
4	ResConvNet + Adam	99.15 %	Skip connections improve gradient flow
5	ResConvNet + BN	99.12 %	Normalizes activations, still overfits
5	ResConvNet + Dropout	99.29 %	Best model — excellent generalization

Moving from a fully-connected baseline to a CNN cut the parameter count by 5× while matching accuracy. Switching from SGD to Adam accelerated convergence. Adding residual connections pushed test accuracy above 99%. Finally, Dropout regularization gave the best test performance (99.29%) with a test loss even lower than train loss, demonstrating its strong regularizing effect. All experiments were run on Google Colab with GPU (CUDA); total training time across all models was approximately 25 minutes.

Use of generative AI

GitHub Copilot was used as a coding assistant for boilerplate code, plotting utilities, and markdown drafting. All architectural decisions, hyperparameter choices, and written analysis are my own. All AI-generated code was reviewed for correctness.

- ☒ The report generally and transparently details all uses of AI-tools, e.g., as a minimum by explaining how it was used and to what extent.
- ☒ Text or code proposed or modified by an AI has been checked for correctness and that the meaning is as intended.
- ☒ I take full responsibility for the whole text.

A Code

```
1  # -*- coding: utf-8 -*-
2  """
3  Deep Learning 1RT720 -- Hand-in Assignment 2
4  MNIST Classification with PyTorch
5
6  Author: Rafael
7  Running on: Google Colab GPU (CUDA)
8
9  Generative AI disclosure: GitHub Copilot was used as a coding assistant
10 for boilerplate, plotting, and markdown drafting. All architectural
11 decisions, hyperparameter choices, and written analysis are my own.
12 """
13
14 # =====
15 # Setup
16 # =====
17
18 import sys
19
20 # Workaround: flush stale torchvision submodules to avoid C++ op
21 # registration conflicts that can occur when restarting Colab sessions.
22 for mod in [k for k in sys.modules if "torchvision" in k]:
23     del sys.modules[mod]
24
25 try:
26     import torch
27     import torch.nn as nn
28     import torch.nn.functional as F
29     import torch.optim as optim
30     from torchvision import datasets, transforms
31     from torch.utils.data import DataLoader
32     import matplotlib.pyplot as plt
33     import numpy as np
34     import time
35     from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
36 except RuntimeError as e:
37     if "already a kernel registered" in str(e):
38         raise RuntimeError(
39             "torchvision C++ ops conflict detected.\n"
40             "-> Go to Runtime -> Restart session, then run cells from here."
41         ) from None
42     raise
43
44 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
45 print(f"Using device: {device}")
46
47 # Reproducibility
48 torch.manual_seed(42)
49 torch.cuda.manual_seed(42)
50
51 # =====
52 # Data Loading
53 # =====
54
55 transform = transforms.Compose([transforms.ToTensor()])
56
57 train_dataset = datasets.MNIST(root='./data', train=True, download=True, transform=transform)
58 test_dataset = datasets.MNIST(root='./data', train=False, download=True, transform=transform)
59
60
61 def get_loaders(batch_size):
62     """Create train and test DataLoaders.
63
64     Args:
65         batch_size: int, batch size for the training loader.
66
67     Returns:
68         (train_loader, test_loader) tuple of DataLoaders.
69     """
70     train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
71     test_loader = DataLoader(test_dataset, batch_size=1000, shuffle=False)
```

```

72     return train_loader, test_loader
73
74 print(f"Train: {len(train_dataset)} images")
75 print(f"Test: {len(test_dataset)} images")
76
77 # =====
78 # Plotting Function
79 # =====
80
81 def training_curve_plot(title, train_costs, test_costs, train_accs, test_accs,
82                        batch_size, learning_rate, num_epochs, elapsed):
83     """Plot cost and accuracy curves for training and test sets.
84
85     Same signature as utils.training_curve_plot from Hand-in 1.
86
87     Args:
88         title: str, plot title.
89         train_costs, test_costs: lists of per-epoch costs.
90         train_accs, test_accs: lists of per-epoch accuracies (fractions in [0,1]).
91         batch_size: int, batch size used.
92         learning_rate: float, learning rate used.
93         num_epochs: int, total epochs.
94         elapsed: float, training time in seconds.
95
96     Returns:
97         matplotlib Figure object.
98     """
99     epochs = np.arange(1, len(train_costs) + 1)
100    time_str = f"{int(elapsed // 60)}min {elapsed % 60:.0f}sec"
101
102    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))
103    fig.suptitle(
104        f"{title}\n| Batch size: {batch_size} | Learning rate: {learning_rate} | "
105        f"| Number of Epochs: {num_epochs} | Training time: {time_str} |",
106        fontsize=11,
107    )
108
109    # Costs
110    ax1.plot(epochs, train_costs, label=f"Final train cost: {train_costs[-1]:.4f}")
111    ax1.plot(epochs, test_costs, label=f"Final test cost: {test_costs[-1]:.4f}")
112    ax1.set_title("Costs"); ax1.set_xlabel("Epochs"); ax1.set_ylabel("Cost")
113    ax1.legend()
114
115    # Accuracy
116    tr_pct = np.array(train_accs) * 100
117    te_pct = np.array(test_accs) * 100
118    ax2.plot(epochs, tr_pct, label=f"Final train accuracy: {tr_pct[-1]:.2f}%")
119    ax2.plot(epochs, te_pct, label=f"Final test accuracy: {te_pct[-1]:.2f}%")
120    ax2.set_title("Accuracy"); ax2.set_xlabel("Epochs"); ax2.set_ylabel("Accuracy (%)")
121    ax2.legend()
122
123    plt.tight_layout()
124    return fig
125
126 # =====
127 # Training & Evaluation Functions
128 # =====
129
130 @torch.no_grad()
131 def evaluate(model, loader, criterion):
132     """Evaluate model on a DataLoader.
133
134     Args:
135         model: nn.Module, the model to evaluate.
136         loader: DataLoader, data to evaluate on.
137         criterion: loss function.
138
139     Returns:
140         (avg_cost, accuracy_fraction) tuple.
141     """
142    model.eval()
143    total_loss = 0.0
144    correct = 0
145    total = 0
146    for X, y in loader:

```

```

147         X, y = X.to(device), y.to(device)
148         out = model(X)
149         total_loss += criterion(out, y).item() * y.size(0)
150         correct += (out.argmax(1) == y).sum().item()
151         total += y.size(0)
152     return total_loss / total, correct / total
153
154
155 def train(model, train_loader, test_loader, optimizer, num_epochs,
156          batch_size, learning_rate, title=""):
157     """Full training loop with per-epoch evaluation and plotting.
158
159     Args:
160         model: nn.Module, the model to train.
161         train_loader, test_loader: DataLoaders for train/test data.
162         optimizer: torch.optim optimizer.
163         num_epochs: int, number of training epochs.
164         batch_size: int, batch size (for plot annotation).
165         learning_rate: float, learning rate (for plot annotation).
166         title: str, title for the learning curve plot.
167
168     Returns:
169         (model, history, elapsed) where history is a dict with keys
170         'train_cost', 'test_cost', 'train_acc', 'test_acc'.
171     """
172     model.to(device)
173     criterion = nn.CrossEntropyLoss()
174
175     train_costs, test_costs = [], []
176     train_accs, test_accs = [], []
177
178     start = time.time()
179     for epoch in range(1, num_epochs + 1):
180         model.train()
181         batch_losses = []
182         correct = 0
183         total = 0
184
185         for X_batch, y_batch in train_loader:
186             X_batch, y_batch = X_batch.to(device), y_batch.to(device)
187
188             out = model(X_batch)
189             loss = criterion(out, y_batch)
190
191             optimizer.zero_grad()
192             loss.backward()
193             optimizer.step()
194
195             batch_losses.append(loss.item())
196             correct += (out.argmax(1) == y_batch).sum().item()
197             total += y_batch.size(0)
198
199         # Epoch stats
200         train_costs.append(np.mean(batch_losses))
201         train_accs.append(correct / total)
202
203         te_cost, te_acc = evaluate(model, test_loader, criterion)
204         test_costs.append(te_cost)
205         test_accs.append(te_acc)
206
207         if epoch == 1 or epoch % 5 == 0 or epoch == num_epochs:
208             print(f"Epoch {epoch:3d}/{num_epochs} -- "
209                   f"Train: cost={train_costs[-1]:.4f} acc={100*train_accs[-1]:.2f}% | "
210                   f"Test: cost={te_cost:.4f} acc={100*te_acc:.2f}%")
211
212     elapsed = time.time() - start
213     print(f"Done in {elapsed:.1f}s")
214
215     fig = training_curve_plot(
216         title, train_costs, test_costs, train_accs, test_accs,
217         batch_size, learning_rate, num_epochs, elapsed,
218     )
219     plt.show()
220
221     history = {

```

```

222         'train_cost': np.array(train_costs), 'test_cost': np.array(test_costs),
223         'train_acc': np.array(train_accs), 'test_acc': np.array(test_accs),
224     }
225     return model, history, elapsed
226
227
228 def count_parameters(model):
229     """Print layer-wise and total learnable parameter counts.
230
231     Args:
232         model: nn.Module.
233
234     Returns:
235         int, total number of parameters.
236     """
237     print("\nParameter count:")
238     print("-" * 55)
239     total = 0
240     for name, p in model.named_parameters():
241         n = p.numel()
242         total += n
243         print(f" {name:35s} {n:>8,} {list(p.shape)}")
244     print("-" * 55)
245     print(f" {'TOTAL':35s} {total:>8,}\n")
246     return total
247
248 # =====
249 # Exercise 1: Multi-layer Fully-Connected Neural Network
250 # Architecture: [784, 128, 64, 10] + ReLU, SGD, same as HW1 Ex3.
251 # =====
252
253 class FullyConnectedNet(nn.Module):
254     """Fully-connected network: [784, 128, 64, 10] with ReLU.
255
256     Same architecture as Hand-in 1 Exercise 3.
257
258     Input: (B, 1, 28, 28) or (B, 784) MNIST images.
259     Output: (B, 10) raw logits for each digit class.
260     """
261     def __init__(self):
262         super().__init__()
263         self.fc1 = nn.Linear(784, 128)
264         self.fc2 = nn.Linear(128, 64)
265         self.fc3 = nn.Linear(64, 10)
266
267     def forward(self, x):
268         x = x.view(x.size(0), -1)          # flatten
269         x = F.relu(self.fc1(x))
270         x = F.relu(self.fc2(x))
271         return self.fc3(x)                # raw logits
272
273 # --- Exercise 1: Training ---
274 bs1, lr1, ep1 = 64, 0.1, 50
275 loader_tr, loader_te = get_loaders(bs1)
276
277 model_fc = FullyConnectedNet()
278 count_parameters(model_fc)
279 opt1 = optim.SGD(model_fc.parameters(), lr=lr1)
280
281 model_fc, hist_fc, elapsed_fc = train(
282     model_fc, loader_tr, loader_te, opt1,
283     ep1, bs1, lr1,
284     title="Exercise 1 -- FC Network (PyTorch) [784,128,64,10]",
285 )
286
287 # =====
288 # Exercise 2: Multi-layer Convolutional Neural Network
289 # Three conv layers + ReLU + max pooling, then FC classifier.
290 # Trained with SGD.
291 # =====
292
293 class ConvNet(nn.Module):
294     """CNN for MNIST classification.
295
296     Architecture:

```

```

297         Conv(1->8, 3x3, pad1) -> ReLU -> MaxPool(2x2)
298         Conv(8->16, 3x3, pad1) -> ReLU -> MaxPool(2x2)
299         Conv(16->32, 3x3, pad1) -> ReLU -> Flatten -> FC(1568, 10)
300
301     Input: (B, 1, 28, 28) MNIST images.
302     Output: (B, 10) raw logits.
303     """
304     def __init__(self):
305         super().__init__()
306         self.conv1 = nn.Conv2d(1, 8, 3, padding=1)
307         self.conv2 = nn.Conv2d(8, 16, 3, padding=1)
308         self.conv3 = nn.Conv2d(16, 32, 3, padding=1)
309         self.pool = nn.MaxPool2d(2, 2)
310         self.fc = nn.Linear(32 * 7 * 7, 10)
311
312     def forward(self, x):
313         x = self.pool(F.relu(self.conv1(x))) # -> (B, 8, 14, 14)
314         x = self.pool(F.relu(self.conv2(x))) # -> (B, 16, 7, 7)
315         x = F.relu(self.conv3(x))           # -> (B, 32, 7, 7)
316         x = x.view(x.size(0), -1)
317         return self.fc(x)
318
319     # --- Exercise 2(a): Parameter count comparison ---
320     model_cnn = ConvNet()
321     count_parameters(model_cnn)
322
323     fc_total = sum(p.numel() for p in FullyConnectedNet().parameters())
324     cnn_total = sum(p.numel() for p in model_cnn.parameters())
325     print(f"FC network: {fc_total:,} params")
326     print(f"CNN: {cnn_total:,} params")
327     print(f"-> CNN uses {fc_total / cnn_total:.1f}x fewer parameters.")
328
329     # --- Exercise 2(b): Training ---
330     bs2, lr2, ep2 = 64, 0.01, 20
331     loader_tr, loader_te = get_loaders(bs2)
332     opt2 = optim.SGD(model_cnn.parameters(), lr=lr2)
333
334     model_cnn, hist_cnn, elapsed_cnn = train(
335         model_cnn, loader_tr, loader_te, opt2,
336         ep2, bs2, lr2,
337         title="Exercise 2 -- CNN with SGD",
338     )
339
340     # =====
341     # Exercise 3: CNN with Adam Optimizer
342     # Same CNN architecture, Adam (default params, lr=0.001).
343     # =====
344
345     bs3, lr3, ep3 = 64, 0.001, 20
346     loader_tr, loader_te = get_loaders(bs3)
347
348     model_adam = ConvNet()
349     opt3 = optim.Adam(model_adam.parameters(), lr=lr3)
350
351     model_adam, hist_adam, elapsed_adam = train(
352         model_adam, loader_tr, loader_te, opt3,
353         ep3, bs3, lr3,
354         title="Exercise 3 -- CNN with Adam",
355     )
356
357     # =====
358     # Exercise 4: Residual Connections
359     # ResConvNet: CNN with residual blocks after each conv stage.
360     # =====
361
362     class ResidualBlock(nn.Module):
363         """Residual block: two conv+ReLU pairs with a skip connection.
364
365         Computes  $F(x) + x$ , where  $F$  is two  $3 \times 3$  conv+ReLU layers.
366
367         Args:
368             channels: int, number of input/output channels (kept constant).
369
370         Input: (B, channels, H, W).
371         Output: (B, channels, H, W).

```

```

372     """
373     def __init__(self, channels):
374         super().__init__()
375         self.conv1 = nn.Conv2d(channels, channels, 3, padding=1)
376         self.conv2 = nn.Conv2d(channels, channels, 3, padding=1)
377
378     def forward(self, x):
379         out = F.relu(self.conv1(x))
380         out = F.relu(self.conv2(out))
381         return out + x                # residual connection
382
383
384 class ResConvNet(nn.Module):
385     """CNN with residual blocks after each convolutional stage.
386
387     Architecture: 3 stages of (Conv -> ReLU -> ResidualBlock -> MaxPool),
388     followed by a linear classifier. Deeper than ConvNet.
389
390     Input: (B, 1, 28, 28) MNIST images.
391     Output: (B, 10) raw logits.
392     """
393     def __init__(self):
394         super().__init__()
395         # Stage 1
396         self.conv1 = nn.Conv2d(1, 8, 3, padding=1)
397         self.res1 = ResidualBlock(8)
398         self.pool1 = nn.MaxPool2d(2, 2)
399         # Stage 2
400         self.conv2 = nn.Conv2d(8, 16, 3, padding=1)
401         self.res2 = ResidualBlock(16)
402         self.pool2 = nn.MaxPool2d(2, 2)
403         # Stage 3
404         self.conv3 = nn.Conv2d(16, 32, 3, padding=1)
405         self.res3 = ResidualBlock(32)
406         # Classifier
407         self.fc = nn.Linear(32 * 7 * 7, 10)
408
409     def forward(self, x):
410         x = self.pool1(self.res1(F.relu(self.conv1(x)))) # -> (B, 8, 14, 14)
411         x = self.pool2(self.res2(F.relu(self.conv2(x)))) # -> (B, 16, 7, 7)
412         x = self.res3(F.relu(self.conv3(x)))             # -> (B, 32, 7, 7)
413         x = x.view(x.size(0), -1)
414         return self.fc(x)
415
416 # --- Exercise 4: Training ---
417 bs4, lr4, ep4 = 64, 0.001, 20
418 loader_tr, loader_te = get_loaders(bs4)
419
420 model_res = ResConvNet()
421 count_parameters(model_res)
422 opt4 = optim.Adam(model_res.parameters(), lr=lr4)
423
424 model_res, hist_res, elapsed_res = train(
425     model_res, loader_tr, loader_te, opt4,
426     ep4, bs4, lr4,
427     title="Exercise 4 -- CNN with Residual Connections",
428 )
429
430 # =====
431 # Exercise 5: Improve the CNN with Regularization
432 # Variant A: Batch Normalization    Variant B: Dropout
433 # =====
434
435 # ---- Variant A: Batch Normalization ----
436 class ResConvNetBN(nn.Module):
437     """ResConvNet with BatchNorm2d after every convolutional layer.
438
439     Input: (B, 1, 28, 28) MNIST images.
440     Output: (B, 10) raw logits.
441     """
442     def __init__(self):
443         super().__init__()
444         # Stage 1
445         self.conv1 = nn.Conv2d(1, 8, 3, padding=1)
446         self.bn1 = nn.BatchNorm2d(8)

```

```

447         self.res1_c1 = nn.Conv2d(8, 8, 3, padding=1)
448         self.res1_bn1 = nn.BatchNorm2d(8)
449         self.res1_c2 = nn.Conv2d(8, 8, 3, padding=1)
450         self.res1_bn2 = nn.BatchNorm2d(8)
451         self.pool1 = nn.MaxPool2d(2, 2)
452         # Stage 2
453         self.conv2 = nn.Conv2d(8, 16, 3, padding=1)
454         self.bn2 = nn.BatchNorm2d(16)
455         self.res2_c1 = nn.Conv2d(16, 16, 3, padding=1)
456         self.res2_bn1 = nn.BatchNorm2d(16)
457         self.res2_c2 = nn.Conv2d(16, 16, 3, padding=1)
458         self.res2_bn2 = nn.BatchNorm2d(16)
459         self.pool2 = nn.MaxPool2d(2, 2)
460         # Stage 3
461         self.conv3 = nn.Conv2d(16, 32, 3, padding=1)
462         self.bn3 = nn.BatchNorm2d(32)
463         self.res3_c1 = nn.Conv2d(32, 32, 3, padding=1)
464         self.res3_bn1 = nn.BatchNorm2d(32)
465         self.res3_c2 = nn.Conv2d(32, 32, 3, padding=1)
466         self.res3_bn2 = nn.BatchNorm2d(32)
467         # Classifier
468         self.fc = nn.Linear(32 * 7 * 7, 10)
469
470     def forward(self, x):
471         # Stage 1
472         x = F.relu(self.bn1(self.conv1(x)))
473         r = x
474         x = F.relu(self.res1_bn1(self.res1_c1(x)))
475         x = F.relu(self.res1_bn2(self.res1_c2(x)))
476         x = self.pool1(x + r)
477         # Stage 2
478         x = F.relu(self.bn2(self.conv2(x)))
479         r = x
480         x = F.relu(self.res2_bn1(self.res2_c1(x)))
481         x = F.relu(self.res2_bn2(self.res2_c2(x)))
482         x = self.pool2(x + r)
483         # Stage 3
484         x = F.relu(self.bn3(self.conv3(x)))
485         r = x
486         x = F.relu(self.res3_bn1(self.res3_c1(x)))
487         x = F.relu(self.res3_bn2(self.res3_c2(x)))
488         x = x + r
489         # Classifier
490         x = x.view(x.size(0), -1)
491         return self.fc(x)
492
493     # ---- Variant B: Dropout ----
494     class ResConvNetDropout(nn.Module):
495         """ResConvNet with Dropout2d after pooling and Dropout before FC.
496
497         Args:
498             p_spatial: float, dropout probability for Dropout2d layers.
499             p_fc: float, dropout probability before the FC classifier.
500
501         Input: (B, 1, 28, 28) MNIST images.
502         Output: (B, 10) raw logits.
503         """
504         def __init__(self, p_spatial=0.25, p_fc=0.5):
505             super().__init__()
506             # Stage 1
507             self.conv1 = nn.Conv2d(1, 8, 3, padding=1)
508             self.res1_c1 = nn.Conv2d(8, 8, 3, padding=1)
509             self.res1_c2 = nn.Conv2d(8, 8, 3, padding=1)
510             self.pool1 = nn.MaxPool2d(2, 2)
511             self.drop1 = nn.Dropout2d(p_spatial)
512             # Stage 2
513             self.conv2 = nn.Conv2d(8, 16, 3, padding=1)
514             self.res2_c1 = nn.Conv2d(16, 16, 3, padding=1)
515             self.res2_c2 = nn.Conv2d(16, 16, 3, padding=1)
516             self.pool2 = nn.MaxPool2d(2, 2)
517             self.drop2 = nn.Dropout2d(p_spatial)
518             # Stage 3
519             self.conv3 = nn.Conv2d(16, 32, 3, padding=1)
520             self.res3_c1 = nn.Conv2d(32, 32, 3, padding=1)
521             self.res3_c2 = nn.Conv2d(32, 32, 3, padding=1)

```



```

522         # Classifier
523         self.drop_fc = nn.Dropout(p_fc)
524         self.fc       = nn.Linear(32 * 7 * 7, 10)
525
526     def forward(self, x):
527         # Stage 1
528         x = F.relu(self.conv1(x))
529         r = x
530         x = F.relu(self.res1_c1(x))
531         x = F.relu(self.res1_c2(x))
532         x = self.drop1(self.pool1(x + r))
533         # Stage 2
534         x = F.relu(self.conv2(x))
535         r = x
536         x = F.relu(self.res2_c1(x))
537         x = F.relu(self.res2_c2(x))
538         x = self.drop2(self.pool2(x + r))
539         # Stage 3
540         x = F.relu(self.conv3(x))
541         r = x
542         x = F.relu(self.res3_c1(x))
543         x = F.relu(self.res3_c2(x))
544         x = x + r
545         # Classifier
546         x = x.view(x.size(0), -1)
547         x = self.drop_fc(x)
548         return self.fc(x)
549
550     # --- Exercise 5(b): Training with Batch Normalization ---
551     bs5, lr5, ep5 = 64, 0.001, 20
552     loader_tr, loader_te = get_loaders(bs5)
553
554     model_bn = ResConvNetBN()
555     opt_bn   = optim.Adam(model_bn.parameters(), lr=lr5)
556     model_bn, hist_bn, elapsed_bn = train(
557         model_bn, loader_tr, loader_te, opt_bn,
558         ep5, bs5, lr5,
559         title="Exercise 5 -- CNN + Batch Normalization",
560     )
561
562     # --- Exercise 5(b): Training with Dropout ---
563     model_do = ResConvNetDropout()
564     opt_do   = optim.Adam(model_do.parameters(), lr=lr5)
565     model_do, hist_do, elapsed_do = train(
566         model_do, loader_tr, loader_te, opt_do,
567         ep5, bs5, lr5,
568         title="Exercise 5 -- CNN + Dropout",
569     )
570
571     # =====
572     # Exercise 5(c): Comparison & Confusion Matrix
573     # =====
574
575     def plot_confusion_matrix(model, loader, title="Confusion Matrix"):
576         """Plot confusion matrix for a model on a given DataLoader.
577
578         Args:
579             model: nn.Module, trained model.
580             loader: DataLoader, test data.
581             title: str, plot title.
582         """
583         model.eval()
584         all_preds, all_true = [], []
585         with torch.no_grad():
586             for X, y in loader:
587                 X, y = X.to(device), y.to(device)
588                 all_preds.extend(model(X).argmax(1).cpu().numpy())
589                 all_true.extend(y.cpu().numpy())
590
591         cm = confusion_matrix(all_true, all_preds)
592         fig, ax = plt.subplots(figsize=(10, 8))
593         ConfusionMatrixDisplay(cm, display_labels=range(10)).plot(ax=ax, cmap='Blues')
594         ax.set_title(title, fontsize=13)
595         plt.tight_layout()
596         plt.show()

```

```

597
598
599 def plot_misclassified(model, loader, title="Misclassified Examples"):
600     """Show 10 misclassified images, prioritizing one per true-label class.
601
602     Args:
603         model: nn.Module, trained model.
604         loader: DataLoader, test data.
605         title: str, plot title.
606     """
607     model.eval()
608     all_errors = []
609
610     # 1. Collect all errors in one pass
611     with torch.no_grad():
612         for X, y in loader:
613             X, y = X.to(device), y.to(device)
614             preds = model(X).argmax(1)
615
616             incorrect_mask = preds != y
617             for x_err, y_err, p_err in zip(X[incorrect_mask], y[incorrect_mask], preds[incorrect_mask]):
618                 all_errors.append((x_err.cpu(), y_err.item(), p_err.item()))
619
620     # 2. Select up to 10, one per unique true class first
621     examples = []
622     seen_classes = set()
623
624     for err in all_errors:
625         true_label = err[1]
626         if true_label not in seen_classes:
627             examples.append(err)
628             seen_classes.add(true_label)
629         if len(examples) == 10:
630             break
631
632     # 3. Pad with remaining errors if fewer than 10 classes had errors
633     if len(examples) < 10:
634         for err in all_errors:
635             if not any(err is ex for ex in examples):
636                 examples.append(err)
637             if len(examples) == 10:
638                 break
639
640     # Plotting
641     fig, axes = plt.subplots(2, 5, figsize=(15, 6))
642     fig.suptitle(title, fontsize=14)
643     for idx, ax in enumerate(axes.flatten()):
644         if idx < len(examples):
645             img, true, pred = examples[idx]
646             ax.imshow(img.squeeze(), cmap='gray')
647             ax.set_title(f"True: {true}, Pred: {pred}", color='red', fontsize=10)
648         ax.axis('off')
649     plt.tight_layout()
650     plt.show()
651
652
653     # Compare and select best model
654     bn_acc = hist_bn['test_acc'][-1]
655     do_acc = hist_do['test_acc'][-1]
656     print(f"BatchNorm final test accuracy: {100*bn_acc:.2f}%")
657     print(f"Dropout final test accuracy: {100*do_acc:.2f}%")
658
659     if bn_acc >= do_acc:
660         best_model, best_name = model_bn, "Batch Normalization"
661     else:
662         best_model, best_name = model_do, "Dropout"
663
664     print(f"\nBest model: {best_name}")
665
666     # Confusion matrix
667     plot_confusion_matrix(best_model, loader_te,
668                           title=f"Exercise 5(c) -- Confusion Matrix ({best_name})")
669
670     # --- Exercise 5(d): Misclassified Examples ---
671     plot_misclassified(best_model, loader_te,

```

